Thapar Institute of Engineering and Technology
(Deemed to be University), Patiala

UCS503P – Software Engineering (Project), Session 2026–27 (Odd Semester)

ThreatLens

AI-Code Security Platform

Project Report – Prototype Stage

Module 1: Repository Scan and Fix Report

Group: 3C52
Branch: COE
Semester: 5th

Submitted by

Vidhi Saxena 1024030700
Yashti Mittal 1024030701
Rajat Verma 1024030696

Under the guidance of

Paramveer Sidhu

October 2026

Abstract

AI coding assistants write code quickly, but the code they produce often skips basic security habits: hardcoded passwords and API keys, outdated libraries with known vulnerabilities, and unsafe coding patterns. Good open-source tools exist for each of these problems, but each tool has its own command, its own output format and its own technical vocabulary, so developers must run several tools and interpret several different reports.

ThreatLens is an AI-code security platform that brings these checks together. This report describes the *prototype stage* of the project, which implements Module 1 (Repository Scan and Fix Report). In the prototype, ThreatLens runs three existing scanners – Gitleaks (leaked secrets), OSV-Scanner (vulnerable dependencies) and Semgrep (insecure code patterns) – through one shared `SecurityEngine`. Every result is converted into a single common `Finding` format, duplicates are removed, findings are ranked from CRITICAL to INFO, and each finding is explained in plain language together with its risk and a suggested fix. Secret values are always redacted. The result is delivered as an HTML, JSON or text report. ThreatLens can also connect to GitHub with an access token, let the user choose a repository and branch, take a temporary shallow clone, scan it, and always delete the clone afterwards.

Development followed 19 phases grouped into 5 stages. Stages 1 to 4 (local scanning, three scanners, understanding and reporting findings, and GitHub connection) are complete and form the prototype. Stage 5 (automatic scanning of GitHub changes with pull-request comments and a security score) and the VS Code extension are planned as the next steps. This report presents the problem, the development approach, the requirements, the design with UML diagrams, the implementation, test results, and the plan for the remaining work.

Contents

Abstract ii

1 Introduction 1
- 1.1 Background and Motivation . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives of the Prototype . . . 1
- 1.4 Scope of this Report . . . 2

2 Project Overview 2
- 2.1 Higher-order Goal . . . 2
- 2.2 The Four Modules . . . 2
- 2.3 Reuse of Existing Engines . . . 3
- 2.4 Time-to-value . . . 3

3 Development Approach: 19 Phases in 5 Stages 3
- 3.1 Stage 1 – Get Local Scanning Working (Phases 0–3) . . . 4
- 3.2 Stage 2 – Add All Three Scanners (Phases 4–8) . . . 4
- 3.3 Stage 3 – Understand and Report Findings (Phases 9–12) . . . 4
- 3.4 Stage 4 – Connect ThreatLens to GitHub (Phases 13–15) . . . 5
- 3.5 Stage 5 – Scan GitHub Changes Automatically (Phases 16–19) . . . 5
- 3.6 Weekly Progress . . . 6

4 Requirements 6
- 4.1 Functional Requirements . . . 6
- 4.2 Non-functional Requirements . . . 7

5 System Design 7
- 5.1 Architecture Overview . . . 7
- 5.2 Project Structure . . . 8
- 5.3 The Common Finding Model . . . 8
- 5.4 Scanner Wrappers . . . 9
- 5.5 SecurityEngine . . . 10
- 5.6 Reporting . . . 11
- 5.7 GitHub Integration . . . 11
- 5.8 Security Decisions Built into the Design . . . 12

6 UML Diagrams 12
- 6.1 Use Case Diagram . . . 12

6.2 Use Case Templates . . . 13
6.3 Class Diagram . . . 16
6.4 Sequence Diagram . . . 17
6.5 Activity Diagram . . . 18
6.6 Component Diagram . . . 20
6.7 Deployment Diagram . . . 20

7 Implementation Highlights **21**
7.1 The Scanner Interface . . . 21
7.2 Failure Isolation and Duplicate Removal . . . 21
7.3 Redacting Secrets . . . 22
7.4 Safe Deletion of the Temporary Clone . . . 22
7.5 The One-call GitHub Scan . . . 22
7.6 Command-line Usage . . . 23

8 Testing and Results **23**
8.1 Test Strategy . . . 23
8.2 Result on the Test Repository . . . 23
8.3 A Sample Finding . . . 24
8.4 Checking the GitHub Integration . . . 24

9 Challenges and Solutions **25**

10 Progress against the Proposal **26**

11 Future Work **26**
11.1 Stage 5: Automatic GitHub Scanning (Phases 16–19) . . . 26
11.2 VS Code Extension (Module 2) . . . 27
11.3 Prompt to Fix the Problem . . . 27
11.4 Further Improvements . . . 27

12 Team Contribution **28**

13 Conclusion **28**

A JSON Report Format **29**

B Setup of the Environment **29**

List of Figures

1 Layered architecture of the ThreatLens prototype 8
2 Use case diagram of the ThreatLens prototype 13
3 Class diagram: scanners, engine, report and finding 16
4 Class diagram: GitHub integration 17
5 Sequence diagram: scanning a GitHub repository (UC2) 18
6 Activity diagram with swimlanes: one ThreatLens run 19
7 Component diagram of the ThreatLens prototype 20
8 Deployment diagram of the prototype 21

List of Tables

1 ThreatLens modules and their status 2
2 What is reused and what ThreatLens adds 3
3 Stages and phases of the development plan 4
4 Progress by week (from the team journals) 6
7 Fields of the `Finding` class 9
8 Severity scale 9
9 The three scanner wrappers 10
10 Security-related design decisions 12
11 Use cases of the prototype 13
12 Use case template: UC1 Scan Local Folder 14
13 Use case template: UC2 Scan GitHub Repository 14
14 Findings by tool and severity on `test-repo` 23
15 Vulnerable dependencies found by OSV-Scanner, by package 24
16 Sample finding as shown in the report 24
17 Problems met during the prototype and how they were solved 25
18 Module 1 goals from the proposal and their prototype status 26
19 Contribution of each team member 28

1 Introduction

1.1 Background and Motivation

The idea for ThreatLens came from something we noticed first-hand. A friend had built and deployed a website, and while looking at it we opened the browser inspector out of curiosity. Some API tokens were visible in the network requests, and with them it was possible to reach other users' data. We were not even looking for a problem; it simply turned up during normal use of a live site.

That incident made us think about how common such mistakes are, especially now that so much code is written with the help of AI tools. AI assistants generate working code very quickly, but the generated code may contain hardcoded credentials, unsafe practices, vulnerable dependencies or missing access control, because nobody asked the assistant to be careful about them.

1.2 Problem Statement

Existing security tools are good at their individual jobs but are hard to use together. The main pain points identified in our proposal are:

- **Too many separate tools:** one tool finds leaked secrets, another finds vulnerable libraries and a third finds insecure code patterns. Developers have to run and understand all of them separately.
- **Problems are found too late:** security mistakes are often discovered only after the code has been pushed, merged or deployed.
- **Reports are hard to understand:** raw scanner output is full of technical terms, and it is not clear what went wrong or how to fix it.
- **Repositories become messy:** old and unused files make it harder to see what actually matters (addressed in a later module).

1.3 Objectives of the Prototype

The prototype stage has one clear aim, in line with our time-to-value approach: *get one complete path working end to end* before adding more features. Its objectives are to:

1. run Gitleaks, OSV-Scanner and Semgrep automatically from Python on a local folder;
2. convert the three different output formats into one common finding format;
3. remove duplicate findings and rank the rest by severity;
4. explain every finding in plain words: what is wrong, why it is dangerous and how to fix it;
5. produce a readable report (HTML, JSON or text) without ever exposing the actual secret;

6. connect to GitHub, select a repository and branch, scan it safely in a temporary clone and delete the clone afterwards.

1.4 Scope of this Report

This report covers **Module 1 – Repository Scan and Fix Report**, stages 1 to 4 of the 19-phase plan. Posting the result as a pull-request comment (Stage 5) and the VS Code extension are not part of the prototype; they are described in Section 11 as the next steps.

2 Project Overview

2.1 Higher-order Goal

The long-term goal is to make AI-written code as safe as carefully hand-written code, without slowing developers down. ThreatLens does this by placing security checks at every point where code enters the picture, with all checks sharing one rule book and one engine.

2.2 The Four Modules

As defined in the proposal, ThreatLens has four modules, each attached to a different moment of the developer's workflow. All of them reuse the same engine and the same finding format. Implementation is done one module at a time, starting with Module 1.

Table 1: ThreatLens modules and their status

Module	Entry point	Purpose	Status
1 – Repo Scan and Fix Report	GitHub repository	Scan the repository for secrets, vulnerable libraries and unsafe code; explain each problem and suggest a fix.	Prototype done (stages 1–4); stage 5 next
2 – Editor Check	IDE / editor	Check new code as it is written, before it is pushed.	Planned (VS Code extension)
3 – Prompt Hardening	AI prompt	Add safety instructions to the prompt before code is generated.	Planned
4 – Deep Scheduled Scan	Scheduler	Periodic deeper scan plus suggestions to clean up unused files and outdated documents.	Planned

2.3 Reuse of Existing Engines

This is an engineering project, not a research project, so the detection itself is done by mature open-source tools. Our own work goes into combining their results, ranking them, explaining them and connecting them to GitHub.

Table 2: What is reused and what ThreatLens adds

Need	**Existing tool reused**	**ThreatLens contribution**
Leaked keys and passwords	Gitleaks	Wrapper, redaction of the secret, severity mapping, plain-language explanation
Vulnerable libraries	OSV-Scanner	Wrapper, CVSS-to-severity mapping, upgrade advice
Unsafe code patterns	Semgrep (`-config auto`)	Wrapper, category and severity mapping, risk text with OWASP/CWE information
GitHub access	PyGithub library and the `git` command	Token login, repository and branch selection, safe temporary clone and clean-up
Combining the results	–	Common `Finding` model, `SecurityEngine`, duplicate removal, sorting, reports

2.4 Time-to-value

We work in short weekly cycles. In each cycle one small, testable piece is added, tried on a controlled test repository and fixed before the next piece is started. Success for the first round was defined simply: one check (a leaked secret) must actually be caught and shown in a report that an ordinary developer can understand and act on.

3 Development Approach: 19 Phases in 5 Stages

The work was planned as 19 small phases, grouped into 5 stages. Each phase has one narrow goal that can be tested on its own, and each stage ends with something that works and can be demonstrated.

Table 3: Stages and phases of the development plan

Stage	Phases	Goal	Status
1	0–3	Get local scanning working (install tools, test Gitleaks, run it from Python, read its output)	Done
2	4–8	Add all three scanners under one engine, with one common format, normalisation and duplicate removal	Done
3	9–12	Understand and report the findings: severity, explanations, report, local testing	Done
4	13–15	Connect ThreatLens to GitHub: login, select repository, scan the repository	Done
5	16–19	Scan GitHub changes automatically: webhooks, pull-request/push scan, post a comment on the pull request, security score	Planned

3.1 Stage 1 – Get Local Scanning Working (Phases 0–3)

- **Phase 0:** install the tools and all requirements, and check them with `gitleaks version`, `osv-scanner -version` and `semgrep -version`.
- **Phase 1:** test Gitleaks manually on a small controlled repository containing fake credentials.
- **Phase 2:** make Python run Gitleaks using `subprocess`.
- **Phase 3:** make Python understand Gitleaks' JSON output and return structured, redacted results.

3.2 Stage 2 – Add All Three Scanners (Phases 4–8)

- **Phase 4:** create one common finding format (the `Finding` class).
- **Phase 5:** build the `SecurityEngine` that runs scanners and collects their findings.
- **Phase 6:** add OSV-Scanner.
- **Phase 7:** add Semgrep.
- **Phase 8:** normalise the results and remove duplicates.

The planned engine flow was: run Gitleaks, run OSV, run Semgrep, collect everything, remove duplicates, sort by severity.

3.3 Stage 3 – Understand and Report Findings (Phases 9–12)

- **Phase 9:** decide the severity of each finding (CRITICAL to INFO).

- **Phase 10:** explain the problem: what is wrong, why it is dangerous and how to fix it.
- **Phase 11:** generate the report.
- **Phase 12:** test everything locally with a single command.

Our planning notes also mark an idea for this stage: along with the explanation, the report should give the developer a ready-made *prompt* that can be handed to an AI assistant to fix the problem. This is planned together with the VS Code extension (Section 11).

3.4 Stage 4 – Connect ThreatLens to GitHub (Phases 13–15)

- **Phase 13:** connect to GitHub using an access token.
- **Phase 14:** select a repository (and branch).
- **Phase 15:** scan the GitHub repository: clone, remove junk folders, scan, delete the clone.

3.5 Stage 5 – Scan GitHub Changes Automatically (Phases 16–19)

- **Phase 16:** webhooks – GitHub sends ThreatLens an HTTP request when something happens.
- **Phase 17:** scan when a pull request is raised or code is pushed.
- **Phase 18:** send the result back to GitHub as a comment on the pull request.
- **Phase 19:** compute a security score.

3.6 Weekly Progress

Table 4: Progress by week (from the team journals)

Week	Stage / phases	Work done
1	Planning	Problem identification, idea selection, brainstorming, division into four modules, choice of Module 1 first, research on open-source tools, initial architecture, common finding format idea, proposal and presentation
2	Stage 1 (0–3)	Tools installed and verified; controlled `test-repo` created; Gitleaks run from Python; JSON output parsed; secrets redacted
3	Stage 2 (4–8)	Common `Finding` model; `SecurityEngine`; OSV-Scanner and Semgrep wrappers; normalisation and duplicate removal
4	Stage 3 (9–12)	Severity classification; explanation, risk and fix text; report generator; single-command CLI; end-to-end test
5 onwards	Stage 4 (13–15)	GitHub login, repository and branch selection, shallow clone, junk-folder removal, scan, guaranteed clean-up; HTML and JSON reports

4 Requirements

4.1 Functional Requirements

ID	Requirement
FR1	The user can scan a local folder with a single command.
FR2	The system runs Gitleaks, OSV-Scanner and Semgrep on the target and collects their results.
FR3	Results from all scanners are converted into one common `Finding` format (tool, rule, category, severity, file, line, description, risk, recommendation, reference, fingerprint).
FR4	Every finding gets one of five severities: CRITICAL, HIGH, MEDIUM, LOW, INFO.
FR5	Duplicate findings are removed and the list is sorted with the most severe first.
FR6	Every finding is explained in plain language: what is wrong, why it matters and how to fix it.
FR7	The report can be produced as HTML (default), JSON (for tools and the future extension) or plain text.
FR8	The actual value of a secret is never shown, logged or stored.
FR9	The user can log in to GitHub with an access token.

ID	Requirement
FR10	The user can select a repository (as `owner/repo` or as a GitHub URL) and a branch.
FR11	The selected repository is cloned into a temporary folder, scanned, and the folder is always deleted afterwards.
FR12	The program ends with an exit code that scripts can use: 0 (no HIGH/CRITICAL), 1 (HIGH or CRITICAL found), 2 (could not run).
FR13	ThreatLens never changes the scanned code by itself; it only suggests fixes for a person to apply.

4.2 Non-functional Requirements

Quality	Requirement
Security	Secrets are redacted before leaving the scanner wrapper; the GitHub token is passed to `git` through an HTTP header in environment variables, never in the URL; everything shown in HTML is escaped; delete operations are restricted to the clone folder.
Reliability	One failing scanner must not stop the others; every external command has a time limit (Gitleaks 120 s, OSV-Scanner 120 s, Semgrep 180 s, clone 300 s).
Extensibility	A new scanner can be added by implementing one method, `scan(repo_path)`, and registering it with the engine.
Usability	Reports use plain words and give a clear next step for every finding.
Performance	Shallow clones (latest snapshot only); repositories above 500 MB are refused; installed libraries and caches are skipped.
Portability	Runs on ordinary laptops with Python 3; developed and tested on Windows, with Linux-compatible commands.
Maintainability	The JSON report carries a `schema_version`, so future programs such as the VS Code extension can rely on a stable contract.

5 System Design

5.1 Architecture Overview

ThreatLens is built in layers. The command-line program is the single entry point. It calls the `SecurityEngine`, which runs the scanner wrappers, each of which calls one external tool. The reporting layer turns the engine's result into a report, and the GitHub integration layer sits in front of the engine to fetch a repository safely.

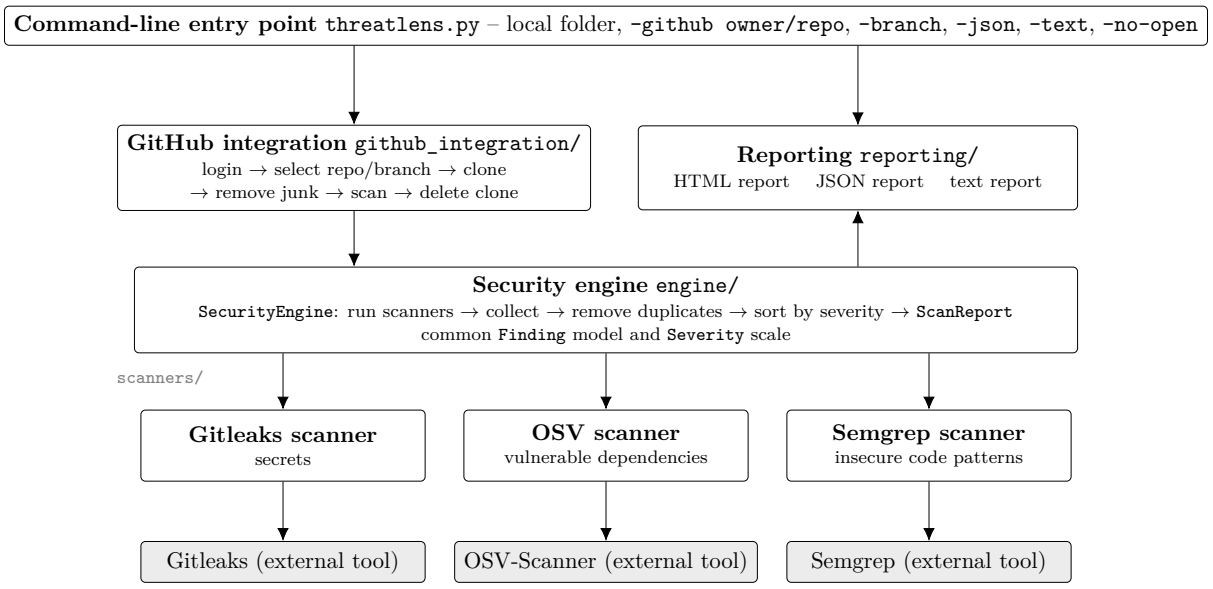


Figure 1: Layered architecture of the ThreatLens prototype

5.2 Project Structure

The source code of the prototype lives in the folder `project a`:

Listing 1: Folder structure of the prototype

```
project a/
|-- threatlens.py              main command-line entry point
|-- engine/
|   |-- finding.py             Finding dataclass and Severity enum
|   `-- security_engine.py     SecurityEngine, ScannerResult, ScanReport
|-- scanners/
|   |-- gitleaks_scanner.py    secrets
|   |-- osv_scanner.py         vulnerable dependencies
|   `-- semgrep_scanner.py     insecure code patterns
|-- reporting/
|   |-- report_generator.py    plain-text report
|   |-- html_report.py         HTML report
|   `-- json_report.py         JSON report (contract for the extension)
|-- github_integration/
|   |-- client.py              token login
|   |-- repos.py, repo_selector.py, branches.py
|   |-- cloner.py              shallow clone
|   |-- cleanup.py             guaranteed deletion of clones
|   |-- repo_scanner.py        junk removal + scan + relative paths
|   `-- pipeline.py            scan_github_repo(): the whole flow
|-- test-repo/                 controlled repo: fake secrets, old libraries
|-- tests/
|-- run_phase1.py, run_phase5.py, run_stage2.py   per-stage demos
|-- requirements.txt
`-- .gitignore
```

5.3 The Common Finding Model

Gitleaks, OSV-Scanner and Semgrep each describe problems in a different way. To combine them, every result is converted into one `Finding` object (a Python dataclass).

Table 7: Fields of the `Finding` class

Field	Type	Meaning
`tool`	text	Gitleaks, OSV-Scanner or Semgrep
`rule`	text	Rule or vulnerability id, e.g. `github-pat`, `CVE-2022-22817`
`category`	text	Secret, Dependency Vulnerability, Code Security, etc.
`severity`	`Severity`	CRITICAL, HIGH, MEDIUM, LOW or INFO
`file`, `start_line`, `end_line`, `start_column`, `end_column`	text, integers	Where the problem is
`description`	text	What is wrong?
`risk`	text	Why is it dangerous?
`recommendation`	text	How should it be fixed?
`reference`	text	Link to a CVE page or rule documentation
`fingerprint`	text	Unique id used for duplicate removal

Table 8: Severity scale

Severity	Meaning
CRITICAL	Production secret exposed, private key, or CVSS score 9.0 or higher
HIGH	API key or access token, CVSS 7.0–8.9
MEDIUM	Generic credential pattern, CVSS 4.0–6.9
LOW	Informational code smell, CVSS below 4.0
INFO	Best-practice suggestion with no immediate risk

5.4 Scanner Wrappers

Each wrapper has one public method, `scan(repo_path)`, which returns a list of `Finding` objects. Because the engine only needs this method, a new scanner can be added without changing the engine.

Table 9: The three scanner wrappers

Wrapper	Detects	Command used	Severity rule
Gitleaks	Hardcoded secrets	`gitleaks detect -source <path> -report-format json -report-path <file>`; `-no-git` is added when the folder has no `.git`	Lookup table by rule id: private keys CRITICAL; specific tokens (GitHub, AWS, Slack, Stripe, ...) HIGH; generic passwords and keys MEDIUM
OSV-Scanner	Dependencies with known vulnerabilities	`osv-scanner scan -format json -recursive <path>`	CVSS score: 9.0 or more CRITICAL, 7.0 or more HIGH, 4.0 or more MEDIUM, otherwise LOW
Semgrep	Insecure code patterns	`semgrep scan -config auto -json -quiet -no-git-ignore <path>`	Semgrep level: ERROR → HIGH, WARNING → MEDIUM, INFO → LOW, NOTE → INFO

Two details matter in practice. First, Gitleaks, OSV-Scanner and Semgrep return exit code 1 when they *find* something; this is not an error, so the wrappers accept exit codes 0 and 1 and treat anything else as a real failure. Second, each wrapper writes the plain-language *description*, *risk* and *recommendation* text itself, so that the final report reads the same whichever tool produced the finding.

5.5 SecurityEngine

The `SecurityEngine` is the central coordinator. Callers only need to know about the engine, not about which scanners exist or how they work.

1. **Register** scanners with `add_scanner(name, scanner)`. The engine checks that the object really has a `scan` method (a `Scanner` protocol).
2. **Run** every scanner. Each run is wrapped in a `try/except`, so a scanner that crashes is recorded as failed (with its error message) and the others continue.
3. **Collect** the findings of all successful scanners.
4. **Remove duplicates** using the `fingerprint` field: the first finding with a given fingerprint is kept.
5. **Sort** by severity (CRITICAL first), then by file and line.
6. **Return** a `ScanReport` holding the findings, the per-scanner results, the total, a count per severity and the list of failed scanners.

5.6 Reporting

- **HTML report (default):** one self-contained page with a summary table, a list of all findings and, for each finding, the sections *What is wrong*, *Why it matters*, *How to fix* and *Reference*. Every value from the scanned repository (file names, messages) is HTML-escaped, because it comes from an unknown source. The page has print rules so that *Print* → *Save as PDF* gives a clean PDF.
- **JSON report (`-json`):** one JSON object on standard output with `schema_version`, `status`, `summary`, `total`, the status of each scanner and the list of findings with paths relative to the scanned folder. It is the contract with the future VS Code extension. If the scan cannot run, a JSON error object with `status: error` is printed instead.
- **Text report (`-text`):** a full report printed in the terminal.

5.7 GitHub Integration

The whole GitHub scan is available as one function, `scan_github_repo(repo, branch)`. It never asks the user anything and never prints, so a server (Stage 5) can call it safely; interactive questions such as "which branch?" are handled only in the command-line program.

1. **Log in** (`client.py`): a fine-grained personal access token is read from the `GITHUB_TOKEN` environment variable or a `.env` file, and one cheap API call proves that it works. Only this file knows how login is done, so a later switch to a GitHub App changes one file.
2. **Select the repository** (`repo_selector.py`): accepts `owner/repo` or common GitHub URL forms, checks the name format, asks GitHub for the repository and refuses empty repositories and those above 500 MB.
3. **Select the branch** (`branches.py`): lists branches (default branch first) and, in interactive use, shows a numbered menu.
4. **Clone** (`cloner.py`): `git clone -depth 1 -single-branch` into a unique temporary folder; the exact commit id is recorded.
5. **Remove junk** (`repo_scanner.py`): folders such as `node_modules`, `site-packages`, caches and any Python virtual environment (detected by its `pyvenv.cfg` file) are deleted from the *copy* only, to avoid slow scans and hundreds of false alarms from other people's code.
6. **Scan:** the same `SecurityEngine` is used, unchanged; absolute file paths are converted to repository-relative paths.
7. **Always delete the clone** (`cleanup.py`): the clone is created in a `with` block (`temporary_clone`), so it is deleted on success, on error and on Ctrl+C. Clones left behind by a hard shutdown are removed on the next run if they are older than 24 hours.

5.8 Security Decisions Built into the Design

Table 10: Security-related design decisions

Decision	Reason
Redact secrets (keep at most 4 characters, e.g. `ghp_***`)	A security tool must not become a new place where secrets leak.
Token sent in an HTTP header through environment variables	The token never appears in the clone URL, in `.git/config` or on the command line, and error messages have it masked.
Delete only inside the clone folder	A bug that passes the wrong path can never delete the user's own files (`remove_tree` refuses any path that is not strictly inside the clone folder).
Suggest fixes, never apply them	Automatic changes could break working code; a person decides.
HTML-escape all scanned values	A file named like a script tag must not run in the reader's browser.

6 UML Diagrams

This section models the prototype with UML diagrams, following the notation taught in the course: the use case diagram with its use case templates, the class diagram, the sequence diagram, the activity diagram, the component diagram and the deployment diagram. The diagrams describe what is implemented in the prototype (Module 1, stages 1–4). The relationships are written as «includes» and «extends», and merges in the activity diagram use merge diamonds (UML 2).

6.1 Use Case Diagram

Actors. The *Developer* is the primary actor: the person who runs ThreatLens. *GitHub* and the *Scanner tools* (Gitleaks, OSV-Scanner, Semgrep) are external systems that ThreatLens uses; they are outside the system boundary.

Table 11: Use cases of the prototype

ID	Use case	Description
UC1	Scan Local Folder	Scan a folder on the computer with all three scanners and produce a report.
UC2	Scan GitHub Repository	Scan one branch of a GitHub repository in a temporary clone and produce a report.
UC3	Run Security Scanners	Run Gitleaks, OSV-Scanner and Semgrep, remove duplicates and sort the findings (included by UC1 and UC2).
UC4	Authenticate with GitHub	Log in with the access token and check that it works (included by UC2 and UC6).
UC5	Clone Repository	Take a temporary shallow clone, remove junk folders, delete the clone after the scan (included by UC2).
UC6	List Branches	Show the branches of a repository.
UC7	Select Branch	Let the user choose a branch when none was given (extends UC2).
UC8	View Security Report	Read the report as HTML, JSON or text.

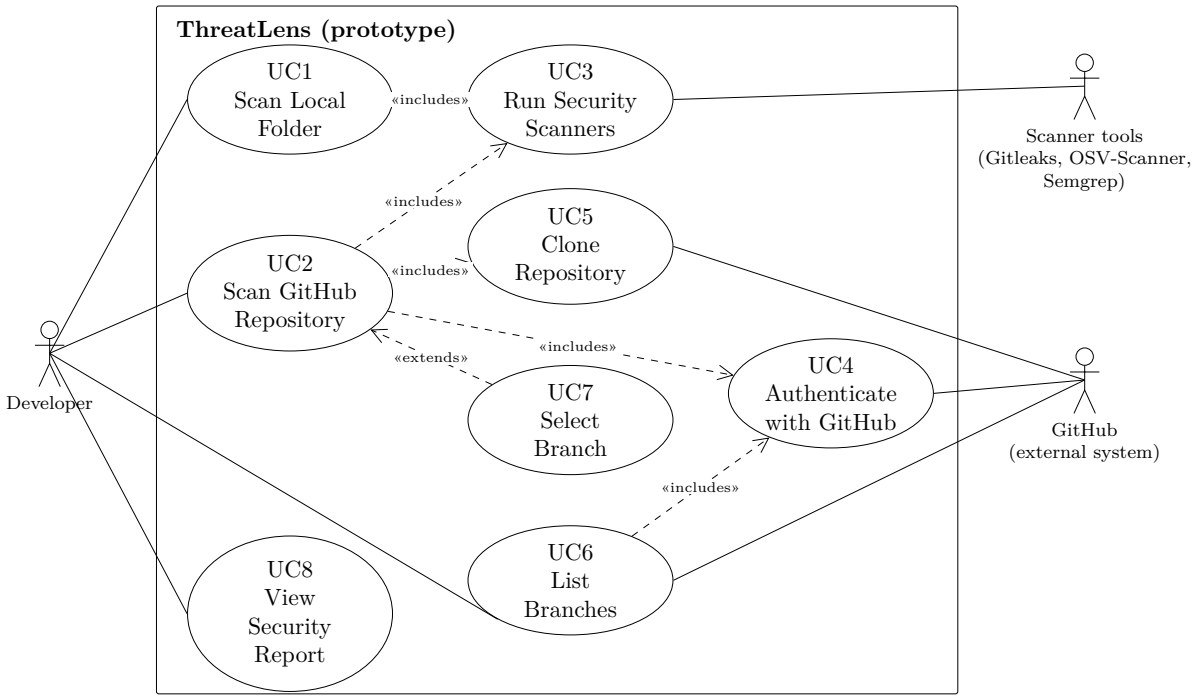


Figure 2: Use case diagram of the ThreatLens prototype

6.2 Use Case Templates

Typical flow notation: AA = actor action, SR = system response.

Table 12: Use case template: UC1 Scan Local Folder

1. Use Case Title	Scan Local Folder
2. Abbreviated Title	Local Scan
3. Use Case Id	UC1
4. Actors	Developer (primary); Scanner tools (external systems)
5. Description	The developer scans a folder on his or her own computer. ThreatLens runs all three scanners and gives one report with plain-language explanations.
5.1 Pre-conditions	The folder exists. Gitleaks, OSV-Scanner and Semgrep are installed.
5.2 Task Sequence (normal flow)	1. (AA) The developer runs `python threatlens.py ./folder`. 2. (SR) The system checks that the folder exists. 3. (SR) The system runs Gitleaks, OSV-Scanner and Semgrep one after another. 4. (SR) The system removes duplicate findings and sorts them by severity. 5. (SR) The system writes the HTML report, opens it in the browser and prints a short summary. 6. (AA) The developer reads the report and fixes the problems.
5.3 Alternate flow	5a. With `-json` the system prints one JSON object; with `-text` it prints the full text report in the terminal; with `-no-open` the browser is not opened.
5.4 Exceptional flow	2a. The folder does not exist: the system shows an error and ends with exit code 2. 3a. A scanner fails or is not installed: the failure is recorded, the other scanners still run, and the report says that results are incomplete.
5.5 Post-conditions	A report is available. The scanned code is unchanged. Exit code is 0 (no HIGH/CRITICAL issues) or 1 (HIGH or CRITICAL issues found).
6. Modification History	13-Oct-2026
7. Author	ThreatLens team, Group 3C52

Table 13: Use case template: UC2 Scan GitHub Repository

1. Use Case Title	Scan GitHub Repository
2. Abbreviated Title	GitHub Scan
3. Use Case Id	UC2

4. Actors	Developer (primary); GitHub and Scanner tools (external systems)
5. Description	The developer scans one branch of a GitHub repository without downloading it manually. ThreatLens clones it temporarily, scans it, deletes the clone and reports the findings.
5.1 Pre-conditions	A valid GitHub access token is in `GITHUB_TOKEN` or in the `.env` file. `git` and the three scanners are installed. The token can access the repository.
5.2 Task Sequence (normal flow)	1. (AA) The developer runs `python threatlens.py -github owner/repo`. 2. (SR) The system reads the token, logs in to GitHub and shows the user name. 3. (SR) The system looks up the repository and checks that it is not empty and not larger than 500 MB. 4. (SR) The system lists the branches (default first) and asks which one to scan. 5. (AA) The developer chooses a branch or presses Enter for the default branch. 6. (SR) The system makes a shallow clone of the branch in a unique temporary folder and records the commit id. 7. (SR) The system removes junk folders (installed libraries, virtual environments, caches) from the copy. 8. (SR) The system runs the three scanners, removes duplicates, sorts by severity and makes file paths relative to the repository. 9. (SR) The system deletes the temporary clone. 10. (SR) The system writes the report (repository, branch and commit are shown) and prints a short summary. 11. (AA) The developer reads the report and fixes the problems.
5.3 Alternate flow	4a. If `-branch name` is given, or the run is not interactive, or `-json` is used, no question is asked and the given or default branch is scanned. 10a. `-json`, `-text` and `-no-open` change the output as in UC1.
5.4 Exceptional flow	2a. The token is missing, mistyped, expired or revoked: error message, exit code 2. 3a. The repository is not found, the token has no access, or the repository is empty or too large: error message, exit code 2. 6a. The branch does not exist, `git` is missing, or cloning takes longer than 5 minutes: error message, exit code 2. 8a. A scanner fails: the others continue and the report marks the results as incomplete. *. If the user presses Ctrl+C, the temporary clone is still deleted.

5.5 Post-conditions	A report is available. The temporary clone is deleted. The repository on GitHub is unchanged. Exit code is 0, 1 or 2 as above.
6. Modification History	13-Oct-2026
7. Author	ThreatLens team, Group 3C52

6.3 Class Diagram

Figure 3 shows the core of the engine. `SecurityEngine` is made of one or more scanners that follow the `Scanner` interface; the three wrappers realise that interface. The engine produces a `ScanReport`, which is composed of `Finding` objects and `ScannerResult` objects. Every scanner wrapper creates `Finding` objects, and each `Finding` has a `Severity`.

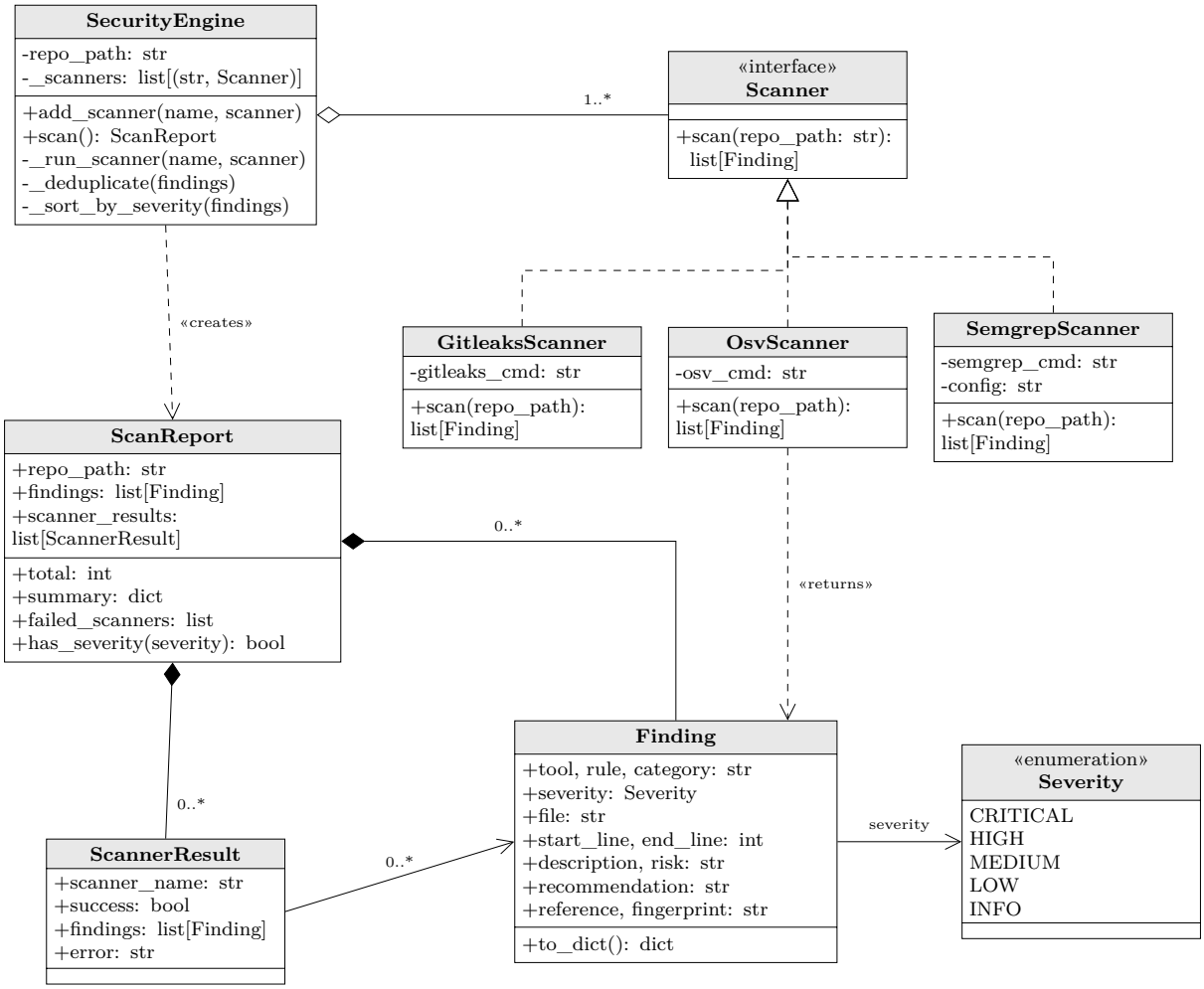


Figure 3: Class diagram: scanners, engine, report and finding

Figure 4 shows the data classes of the GitHub integration. The functions of that package are grouped in modules («module»), not classes: `client`, `repo_selector`, `branches`, `cloner`, `cleanup`, `repo_scanner` and `pipeline`. The single function `scan_github_repo()` in `pipeline` uses the others and returns a `GitHubScanResult`.

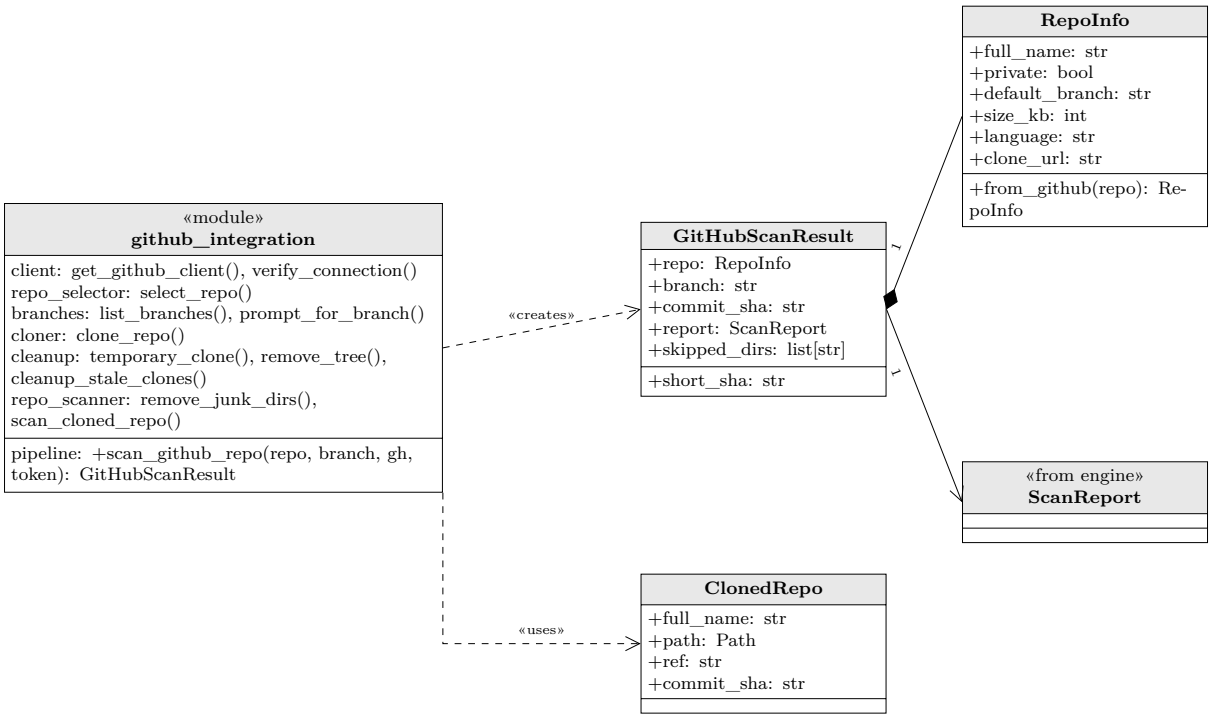


Figure 4: Class diagram: GitHub integration

6.4 Sequence Diagram

Figure 5 shows the objects and messages for the main scenario, scanning a GitHub repository (UC2). Solid arrows are calls, dashed arrows are returns. The branch question (messages 4 and 5) is asked only in interactive use.

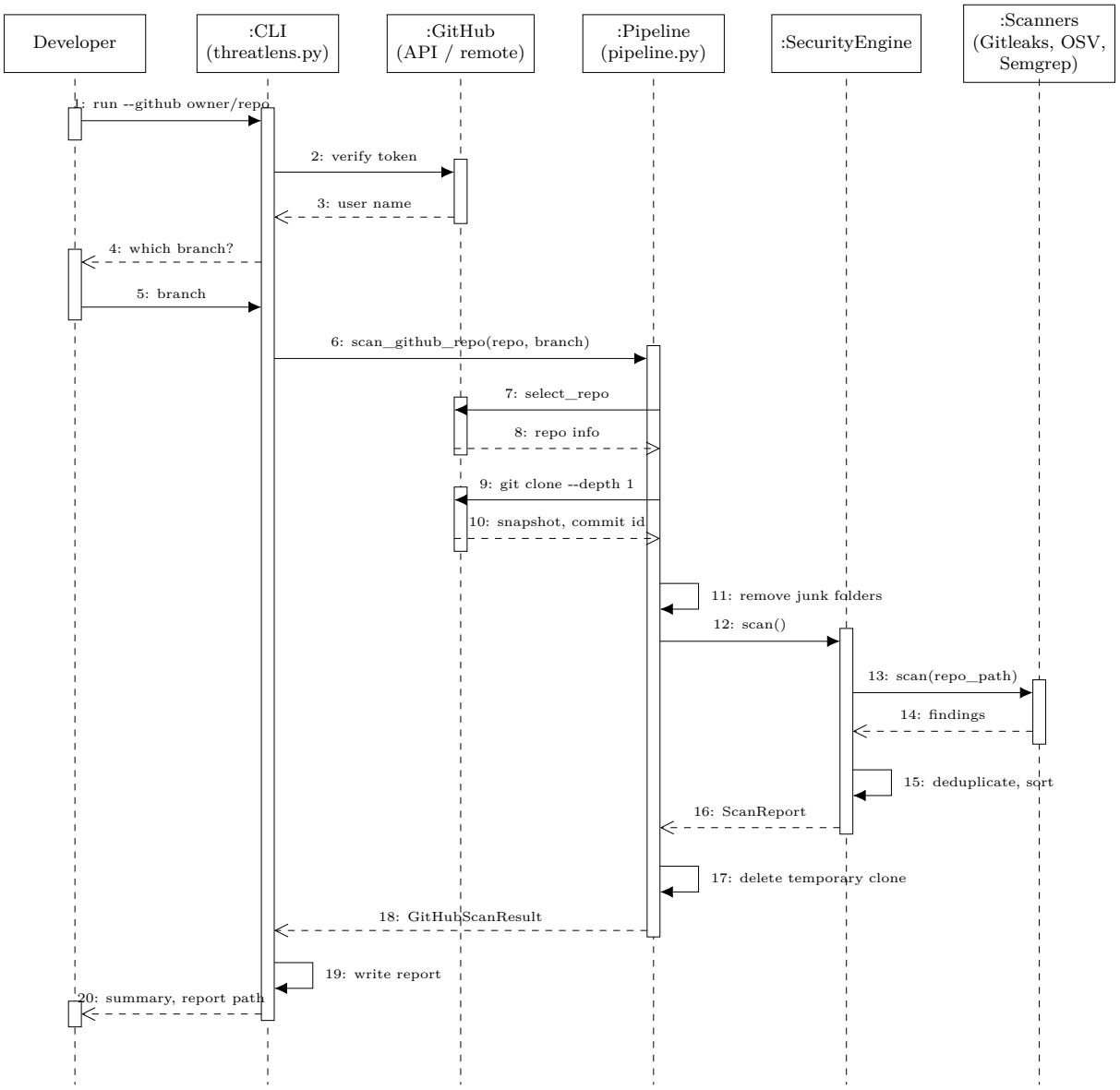


Figure 5: Sequence diagram: scanning a GitHub repository (UC2)

6.5 Activity Diagram

Figure 6 shows the overall flow of one run, with swimlanes for the Developer, ThreatLens and GitHub. The scanners run one after another; a failing scanner is recorded and does not stop the rest. Any failure before scanning (folder missing, bad token, repository not found, clone error) ends the run with an error message and exit code 2.

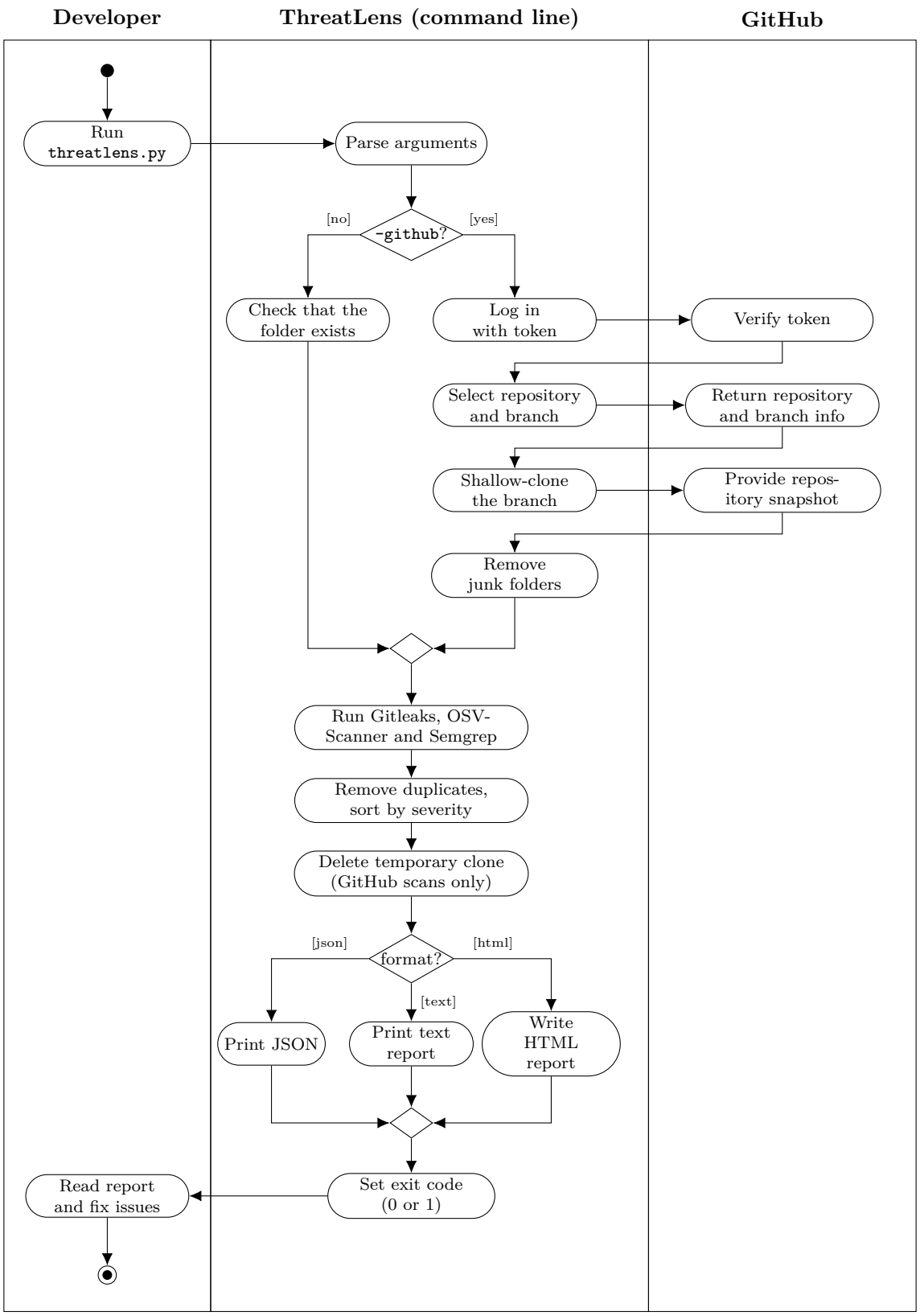


Figure 6: Activity diagram with swimlanes: one ThreatLens run

6.6 Component Diagram

The component diagram shows the physical structure of the implementation: the packages of ThreatLens, the external tools and libraries they depend on, and the artifacts they produce.

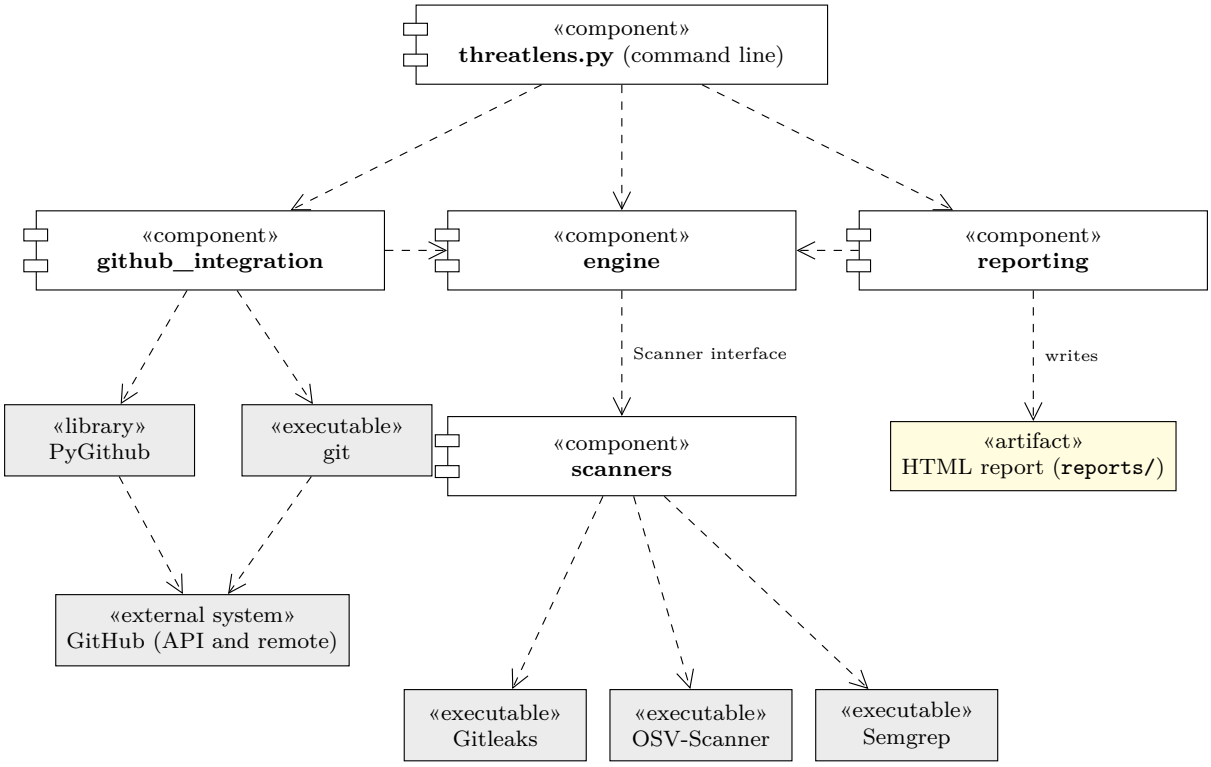


Figure 7: Component diagram of the ThreatLens prototype

6.7 Deployment Diagram

The prototype runs entirely on the developer's own computer; no server is needed yet. The only outside connections are to GitHub and, for the scanners' own data, to the vulnerability database and rule registry that OSV-Scanner and Semgrep use.

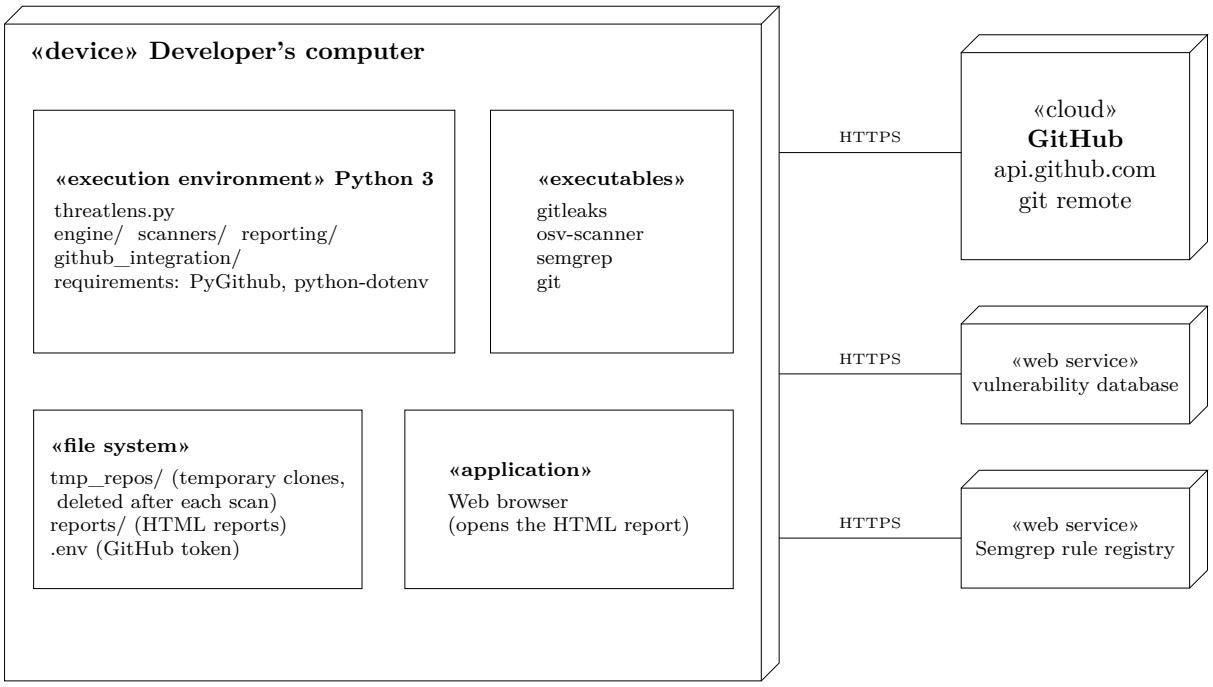


Figure 8: Deployment diagram of the prototype

7 Implementation Highlights

The prototype is written in Python 3 and uses the existing tools as external programs, started with `subprocess`. Only the most important pieces of code are shown here.

7.1 The Scanner Interface

The engine depends only on this small interface. Any class with a `scan` method can be registered, which is how Gitleaks, OSV-Scanner and Semgrep are plugged in the same way.

Listing 2: The `Scanner` protocol (engine/security_engine.py)

```
@runtime_checkable
class Scanner(Protocol):
    def scan(self, repo_path: str) -> list[Finding]:
        """Scan the repository and return a list of Finding objects."""
        ...
```

7.2 Failure Isolation and Duplicate Removal

Each scanner result is wrapped in a `ScannerResult`. If a scanner is missing or crashes, only that result is marked as failed. Duplicates are recognised by their fingerprint, and the first occurrence is kept.

Listing 3: Duplicate removal by fingerprint (simplified)

```
def _deduplicate(self, findings):
    seen_fingerprints = set()
    unique = []
```

```
    for finding in findings:
        if not finding.fingerprint:          # cannot compare, keep it
            unique.append(finding)
        elif finding.fingerprint not in seen_fingerprints:
            seen_fingerprints.add(finding.fingerprint)
            unique.append(finding)
    return unique
```

7.3 Redacting Secrets

Gitleaks reports the real secret value in its JSON file. The wrapper replaces it before creating a `Finding`, so the full value is never stored in memory structures that are later printed or saved.

Listing 4: Secret redaction (scanners/gitleaks_scanner.py)

```
def _redact(value: str) -> str:
    # 'ghp_FakeGithubToken123' -> 'ghp_***'
    if not value or len(value) <= 4:
        return "[REDACTED]"
    return value[:4] + "***"
```

7.4 Safe Deletion of the Temporary Clone

Listing 5: Guard against deleting the wrong folder (github_integration/cleanup.py)

```
def remove_tree(path, allowed_root):
    path = Path(path).resolve()
    root = Path(allowed_root).resolve()
    # a wrong path must never wipe the user's own files
    if path == root or root not in path.parents:
        raise UnsafeDeleteError(f"Refusing to delete {path}")
    ...   # delete with retries (Windows file locks)

@contextmanager
def temporary_clone(repo, token=None, ref=None):
    cloned = clone_repo(repo, token=token, ref=ref)
    try:
        yield cloned
    finally:                      # success, error or Ctrl+C
        remove_tree(cloned.path, allowed_root=get_clone_dir())
```

7.5 The One-call GitHub Scan

Listing 6: Core of `scan_github_repo` (github_integration/pipeline.py)

```
def scan_github_repo(repo, branch=None, gh=None, token=None):
    ...
    info = select_repo(gh, repo)
    with temporary_clone(info, token=token, ref=branch) as cloned:
        report, skipped = scan_cloned_repo(cloned)
        result = GitHubScanResult(repo=info, branch=cloned.ref,
                                  commit_sha=cloned.commit_sha,
                                  report=report, skipped_dirs=skipped)
    # the clone is already deleted here
    return result
```

7.6 Command-line Usage

Listing 7: Main commands

```
python threatlens.py test-repo                      # scan a local folder, open HTML report
python threatlens.py test-repo --text               # print the text report
python threatlens.py test-repo --json               # JSON output (for the extension)
python threatlens.py --github owner/repo            # scan a GitHub repo (asks for branch)
python threatlens.py --github owner/repo --branch dev --no-open
```

8 Testing and Results

8.1 Test Strategy

- **A controlled test repository.** `test-repo/` is a small project that is *deliberately* insecure and is never meant to run. It contains fake credentials (a GitHub token, a Slack webhook URL, AWS-style keys, a password and a database URL) in `app.py` and `config.py`, a `requirements.txt` with intentionally old versions (requests 2.18.0, Pillow 9.0.0, Flask 0.12.2) and two newer packages as a control (python-dotenv 1.0.0, click 8.1.7). Because everything in it is known in advance, the output can be checked by hand.
- **Stage-by-stage checks.** Each stage was verified before the next one started: Gitleaks alone (`run_phase1.py`), all three scanners under one engine (`run_stage2.py`), the engine with severity sorting (`run_phase5.py`), and then the complete command-line run.
- **Each module on its own.** Every GitHub module can be run directly (for example `python -m github_integration.repo_selector owner/repo`), so a problem can be isolated to login, selection, branch listing, cloning or clean-up.

8.2 Result on the Test Repository

Running `python threatlens.py test-repo` on the controlled repository produced 36 findings, sorted from CRITICAL to MEDIUM and shown in the HTML report.

Table 14: Findings by tool and severity on `test-repo`

Tool	CRITICAL	HIGH	MEDIUM	Total
Gitleaks (secrets)	0	2	0	2
Semgrep (code patterns)	0	1	0	1
OSV-Scanner (dependencies)	3	18	12	33
Total	**3**	**21**	**12**	**36**

Table 15: Vulnerable dependencies found by OSV-Scanner, by package

Package (version in test-repo)	Findings	Severities
Pillow 9.0.0	23	3 CRITICAL, 14 HIGH, 6 MEDIUM
Requests 2.18.0	5	1 HIGH, 4 MEDIUM
Flask 0.12.2	4	3 HIGH, 1 MEDIUM
python-dotenv 1.0.0	1	1 MEDIUM

The secret findings were a GitHub personal access token (`app.py`, line 29) and a Slack webhook URL (`config.py`, line 24), both HIGH, with the value redacted in the report. Gitleaks and Semgrep use different detection rules, so each reports the credentials that match its own patterns.

8.3 A Sample Finding

Every finding in the report has the same layout. The first CRITICAL finding is shown below.

Table 16: Sample finding as shown in the report

Heading	CRITICAL – Dependency Vulnerability, `requirements.txt`, OSV-Scanner, rule CVE-2023-50447
What is wrong	Package `pillow` version 9.0.0 has a known vulnerability (CVE-2023-50447): arbitrary code execution in Pillow.
Why it matters	The vulnerability affects a dependency the project imports (CVSS score 9.3). An attacker exploiting it could compromise any application that uses this package.
How to fix	(1) Upgrade `pillow` to the latest patched version. (2) Run `pip install -upgrade pillow`. (3) Update `requirements.txt` or the lock file. (4) Re-run ThreatLens to confirm the vulnerability is gone.
Reference	`https://osv.dev/vulnerability/PYSEC-2026-457`

8.4 Checking the GitHub Integration

The Stage 4 modules are checked with the commands below, each of which prints a clear success or failure message:

Listing 8: Commands used to check each GitHub module

```
python -m github_integration.client                      # token works?
python -m github_integration.repo_selector owner/repo    # repo found and allowed?
python -m github_integration.cloner owner/repo           # shallow clone works?
python -m github_integration.repo_scanner owner/repo     # clone, scan, delete, report
```

9 Challenges and Solutions

Table 17: Problems met during the prototype and how they were solved

Challenge	Solution
The three tools return different formats (JSON structures, field names, severity scales)	One common `Finding` class; each wrapper translates its own output into it.
The scanners exit with code 1 when they *find* something, which looks like a failure	Wrappers accept exit codes 0 and 1 and treat any other code as a real error.
Gitleaks prints the real secret in its output	Redaction inside the wrapper, before a `Finding` is created.
Raw scanner messages are hard to read	Each wrapper builds its own *what is wrong / why it matters / how to fix* text.
One broken or missing tool should not hide the other results	Each scanner runs in its own `try/except`; failed scanners are listed in the report.
Scanners give file paths in different styles (absolute for Gitleaks and OSV, relative for Semgrep)	Paths are converted to repository-relative paths with forward slashes, ready for GitHub and the extension.
A cloned repository can contain a virtual environment or `node\_modules`, causing slow scans and hundreds of false alarms	Junk folders are deleted from the temporary copy before scanning.
Windows cannot delete read-only `.git` files and sometimes keeps files locked briefly	Deletion makes files writable and retries up to three times.
Leaving clones on disk is a security risk	`temporary_clone` always deletes in a `finally` block; clones older than 24 hours are swept on the next run.
Putting the token in a clone URL could leak it into `.git/config` and logs	The token is sent in an HTTP header through environment variables and masked in error messages.
Windows terminals can fail on non-ASCII characters (such as symbols in reports)	Output streams are set to UTF-8.

10 Progress against the Proposal

Table 18: Module 1 goals from the proposal and their prototype status

Goal	Status
Connect to a GitHub repository	Done (token login, repository and branch selection, shallow clone)
Catch leaked secrets, vulnerable libraries and unsafe code patterns	Done (Gitleaks, OSV-Scanner, Semgrep)
Common finding format shared by all engines	Done (`Finding`, `Severity`, `SecurityEngine`)
Remove duplicates and rank by severity	Done
Plain-language explanation and fix for each finding	Done (HTML, JSON and text reports)
Do not expose the secret itself	Done (redaction)
Scan automatically when a pull request is raised or code is pushed	Planned (Stage 5, phases 16–17)
Post the result as a comment on the pull request	Planned (Stage 5, phase 18)
Security score	Planned (Stage 5, phase 19)
Ready-made prompt to give an AI assistant to fix each problem	Planned (with the extension)
Modules 2, 3 and 4	Planned

11 Future Work

11.1 Stage 5: Automatic GitHub Scanning (Phases 16–19)

- **Webhooks (phase 16):** a small web server receives the HTTP request GitHub sends when something happens in a repository. The planned Flask dependency is already listed in `requirements.txt`.
- **Scan on pull request or push (phase 17):** the server calls the existing

`scan_github_repo()` function, which was designed so that it never asks questions and never prints.

- **Post a comment on the pull request (phase 18):** the findings, already stored with repository-relative paths and line numbers, are sent back to GitHub.
- **Security score (phase 19):** a single score summarising the findings of a scan.

11.2 VS Code Extension (Module 2)

The extension will be built in later steps, on top of the `-json` output that already exists. The steps in our planning notes are:

1. create the VS Code extension (TypeScript/JavaScript);
2. connect the extension to the ThreatLens engine;
3. show findings in the VS Code *Problems* (diagnostics) panel;
4. build a ThreatLens sidebar user interface;
5. GitHub integration inside the extension;
6. automatic pull-request/push scanning with comments;
7. production testing, packaging and publishing.

11.3 Prompt to Fix the Problem

Each finding already explains the problem. A planned improvement is to add a ready-made *prompt* to every finding in the report and in the extension, containing the file, line, problem and the safe fix to apply. The developer can give this prompt to an AI assistant, which closes the loop: AI-written code is checked, and the AI can then be asked to repair what the check found.

11.4 Further Improvements

- correlate findings that different scanners report for the same place into a single entry;
- a formal automated test suite with `pytest` for every scanner wrapper and for the report writers;
- Modules 3 and 4: prompt hardening and the scheduled deep scan with repository clean-up suggestions.

12 Team Contribution

Table 19: Contribution of each team member

Member	Roll no.	Main contribution
Vidhi Saxena	1024030700	Tool setup and verification; scanner wrappers; `SecurityEngine` and the combined scanning flow (stages 1–2)
Rajat Verma	1024030696	Findings: severity classification, plain-language explanations and reporting of the findings (stage 3)
Yashti Mittal	1024030701	GitHub integration: login, repository and branch selection, cloning, clean-up and the GitHub scan pipeline (stage 4)

All three members took part in choosing the idea, planning the 19 phases, writing the proposal and testing the prototype.

13 Conclusion

The prototype shows that the central idea of ThreatLens works. One command now runs three different security tools, converts their different outputs into one common format, removes duplicates, ranks the findings and explains each one in plain language, without ever showing the secret itself. The same engine can scan either a local folder or a GitHub repository, which is cloned temporarily and always deleted afterwards. Because the engine, the finding format and the JSON contract are shared, the next steps – automatic pull-request scanning with comments, the VS Code extension and the "prompt to fix" feature – can be added on top without redesigning what exists.

References

[1] Gitleaks – tool for detecting secrets in code. https://github.com/gitleaks/gitleaks

[2] OSV-Scanner and the OSV vulnerability database. https://google.github.io/osv-scanner/ and https://osv.dev

[3] Semgrep – static analysis for code patterns. https://semgrep.dev

[4] PyGithub documentation. https://pygithub.readthedocs.io

[5] GitHub documentation: personal access tokens, REST API and webhooks. https://docs.github.com

[6] OWASP Top 10 and CWE (Common Weakness Enumeration), used in the risk explanations. https://owasp.org/www-project-top-ten/, https://cwe.mitre.org

[7] G. Booch, J. Rumbaugh, I. Jacobson, *The UML User Guide*, Addison-Wesley.

[8] Software Engineering Team, CSED, TIET Patiala: course slide sets on UML (use case, activity, class, sequence, component and deployment diagrams).

A JSON Report Format

The `-json` option prints one JSON object. Its structure (schema version 1) is the contract used by later programs such as the VS Code extension.

Listing 9: Structure of the JSON report

```
{
  "schema_version": 1,
  "status": "ok",
  "scanned_path": "...",
  "repository": "owner/repo",      // GitHub scans: repository, branch, commit
  "summary": { "CRITICAL": 0, "HIGH": 0, "MEDIUM": 0, "LOW": 0, "INFO": 0 },
  "total": 0,
  "scanners": [ { "name": "Gitleaks", "ok": true, "error": null } ],
  "findings": [ { "tool": "...", "rule": "...", "severity": "...", "file": "...",
                  "description": "...", "risk": "...", "recommendation": "..." } ]
}
// If the scan cannot run:
{ "schema_version": 1, "status": "error", "error_type": "...", "message": "..." }
```

B Setup of the Environment

1. Install Python 3 and `git`.
2. Install the three scanners (Gitleaks, OSV-Scanner, Semgrep) and check them with `gitleaks version`, `osv-scanner -version` and `semgrep -version`.
3. Install the Python libraries: `pip install -r requirements.txt`.
4. For GitHub scans, create a fine-grained personal access token on GitHub (read access to the repository contents) and add the line `GITHUB_TOKEN=...` to a `.env` file in the project folder. The `.env` file is excluded from version control by `.gitignore`.